

SSH Tunneling for Modbus/TCP Protocol

Project Team Members:

Pavai Vendhan Ganesan – 22408095

Nandhu Krishna – 12640669

Joyal Joshy – 12630286

A project report presented for the module
**Selected Topics of Embedded Software
Development I**



Embedded Security – MASTER Applied Computer Science
Summer Term 2026

Source Code Repository:

<https://github.com/pavaivendhan/ESD-Modbus-SSH-Tunnel>

Abstract

The reliance of modern critical infrastructure on legacy industrial communication protocols presents a severe security risk. Modbus/TCP, a pervasive protocol utilized in Supervisory Control and Data Acquisition (SCADA) systems and Programmable Logic Controllers (PLCs), was designed in an era prior to the ubiquity of cyber-physical threats. It inherently lacks any mechanisms for authentication, encryption, or cryptographic integrity checking. Consequently, industrial networks utilizing Modbus/TCP are highly vulnerable to eavesdropping, Man-in-the-Middle (MITM) attacks, and unauthorized data injection, potentially leading to catastrophic physical damage.

This report comprehensively explores the theoretical vulnerabilities of the Modbus/TCP protocol and empirically demonstrates these flaws through dynamic network analysis utilizing Wireshark. To remediate this critical vulnerability without necessitating the replacement of legacy hardware, this project proposes and implements a cryptographic mitigation strategy utilizing Secure Shell (SSH) tunneling. By encapsulating plaintext Modbus traffic within an AES-encrypted SSH tunnel, confidentiality, integrity, and authentication are effectively retrofitted into the legacy system.

Furthermore, this report critically evaluates the feasibility of deploying this mitigation on resource-constrained embedded systems. Through automated benchmarking, the cryptographic overhead (CPU utilization and network latency) is quantified. The findings unequivocally demonstrate that SSH tunneling provides a robust, highly effective, and computationally viable security enhancement for existing industrial embedded systems.

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Project Objectives	1
2 Theoretical Background	2
2.1 The Modbus Protocol	2
2.1.1 Data Model	2
2.2 Evolution to Modbus/TCP	2
3 Threat Landscape for Modbus/TCP	3
3.1 Lack of Authentication	3
3.2 Eavesdropping (Loss of Confidentiality)	3
3.3 Data Injection and Manipulation (Loss of Integrity)	3
3.4 Replay Attacks	3
3.5 Historical Precedent: The Stuxnet Context	3
4 Cryptographic Mitigation: SSH Tunneling	4
4.1 The Mechanics of Secure Shell (SSH)	4
4.2 Local Port Forwarding Architecture	4
5 Project Architecture and Implementation	5
5.1 Component Breakdown	5
5.1.1 The Virtual PLC (Server)	5
5.1.2 The Modbus Client	6
5.1.3 The SSH Tunnel Orchestrator	6
5.2 Environment Automation via Makefile	6
6 Implementation Setup and Key Workarounds	8
6.1 Port 5020 vs Port 502 (Privilege Separation)	8
6.2 Client-Side Data Generation	8
7 Execution Phase I: Plaintext Vulnerability Analysis	9
7.1 Base System Execution	9
7.2 Proving the Eavesdropping Vulnerability	10
7.2.1 Vulnerability Confirmation	10
8 Execution Phase II: Secure Tunnel Mitigation	12
8.1 Establishing the Encrypted Tunnel	12
8.2 Routing Client Traffic through the Tunnel	13
8.3 Proving the Cryptographic Solution	14

8.3.1	Solution Confirmation	14
9	Performance Benchmarking and Feasibility	16
9.1	Methodology	16
9.2	Empirical Results and Analysis	17
9.2.1	Latency Overhead Analysis	17
9.2.2	CPU Utilization Analysis	17
10	Future Trends in Embedded System Security	18
10.1	Protocol Migration (Shift-Left Security)	18
10.2	Zero Trust Architecture (ZTA) in OT	18
10.3	Hardware Security Modules (HSMs) and TPMs	18
10.4	AI-Powered Intrusion Detection Systems (IDS)	19
11	Conclusion	20

List of Figures

1	Clean Architecture mapping of the Application, Security, and Network layers.	5
2	Project Makefile Automation	7
3	Terminal output: Initialization of the Virtual PLC Modbus Server. . . .	9
4	Terminal output: The Modbus Client successfully writing and reading Boiler Temperatures and Motor Speeds.	9
5	Wireshark Capture: The plaintext Boiler Temperature fully exposed in the payload.	10
6	Wireshark Capture: The plaintext Motor Speed fully exposed.	11
7	Option A: Establishing the tunnel utilizing the Native macOS SSH Client via command line.	12
8	Option B: Establishing the tunnel utilizing the cross-platform Python <code>sshtunnel</code> library.	12
9	Terminal output: The Modbus Client seamlessly interacting with the server via the encrypted <code>localhost:2222</code> endpoint.	13
10	Wireshark Capture: The outgoing Modbus write request mathematically obscured within an Encrypted SSH Packet.	14
11	Wireshark Capture: The incoming Modbus read response fully encrypted.	15
12	Terminal output: Execution of the 1000-request benchmarking suite. . . .	16
13	Matplotlib visualization contrasting the Latency (left) and CPU (right) overhead of plaintext vs. encrypted Modbus communication.	17

1 Introduction

Industrial Control Systems (ICS) and Supervisory Control and Data Acquisition (SCADA) systems form the digital backbone of modern civilization. They regulate critical infrastructure like power grids, water facilities, nuclear reactors and manufacturing floors. Historically, these systems were physically isolated from public networks via "air-gapping."

However, Industry 4.0 and the Industrial Internet of Things (IIoT) have drastically altered this landscape. The need for real-time data analytics, remote diagnostics, and IT integration has eradicated the traditional air-gap. Today, industrial systems are interconnected, bridging Operational Technology (OT) and Information Technology (IT).

This convergence exposes legacy protocols to modern cyber threats. The most prominent is Modbus. While offering unparalleled simplicity and reliability, it was engineered in 1979 without cybersecurity in mind. As the protocol adapted from serial communication lines to Ethernet (Modbus/TCP), its lack of security controls became a critical liability.

1.1 Problem Statement

Modbus/TCP transmits all control data and sensor readings in absolute plaintext. It requires no authentication to establish a connection, and it provides no cryptographic checksums to verify that a packet has not been altered in transit. If an adversary gains access to the local industrial network, they can easily eavesdrop on sensitive operational data or, far more dangerously, inject malicious control commands directly into the Programmable Logic Controllers (PLCs) governing physical machinery.

1.2 Project Objectives

The primary objective of this project is to deeply analyze the inherent vulnerabilities of Modbus/TCP and to implement a practical, cost-effective mitigation strategy. Specifically, this project will:

1. Establish a virtual testbed consisting of a Modbus/TCP Client and Server (Virtual PLC).
2. Empirically demonstrate the protocol's vulnerability by capturing and analyzing plaintext traffic using Wireshark.
3. Implement a transparent cryptographic wrapper using SSH Tunneling (Port Forwarding) to encapsulate the legacy protocol.
4. Prove the effectiveness of the mitigation by analyzing the encrypted network traffic.
5. Evaluate the performance overhead of the cryptographic encapsulation to determine its feasibility on resource-constrained embedded systems.

2 Theoretical Background

2.1 The Modbus Protocol

Developed by Modicon (now Schneider Electric) in 1979, Modbus is arguably the most widely used industrial communication protocol in the world. Its success is attributed to its open-source nature, simplicity, and ease of deployment.

Modbus operates on a strict Master/Slave architecture (modernized to Client/Server terminology in Ethernet deployments). In this architecture, only the Master (Client) can initiate communication. The Slaves (Servers/PLCs) remain silent on the network until they are directly polled or commanded by the Master.

2.1.1 Data Model

Modbus interacts with four distinct memory areas within a PLC:

- **Discrete Inputs (Read-Only, 1 bit):** Used for reading binary physical sensors (e.g., limit switches, proximity sensors).
- **Coils (Read/Write, 1 bit):** Used for commanding binary physical actuators (e.g., turning a relay or motor on/off).
- **Input Registers (Read-Only, 16 bits):** Used for reading analog physical sensors (e.g., temperature probes, pressure transducers).
- **Holding Registers (Read/Write, 16 bits):** Used for storing configuration variables or commanding analog actuators (e.g., setting a VFD motor speed).

2.2 Evolution to Modbus/TCP

As industrial networks evolved from specialized serial cables (RS-232/RS-485) to standard Ethernet (TCP/IP), the Modbus protocol was adapted to create Modbus/TCP.

In Modbus/TCP, the traditional serial checksum (CRC) is discarded, relying entirely on the underlying TCP/IP stack for basic transmission error checking. A new header, the Modbus Application Protocol (MBAP) header, is prepended to the Modbus Protocol Data Unit (PDU).

The MBAP header is 7 bytes long and contains:

- **Transaction Identifier (2 bytes):** Used to match responses to requests.
- **Protocol Identifier (2 bytes):** Always 0x0000 for Modbus.
- **Length (2 bytes):** The number of following bytes in the message.
- **Unit Identifier (1 byte):** Used for routing to specific devices on a serial sub-network.

Standard Modbus/TCP operates on **TCP Port 502**.

3 Threat Landscape for Modbus/TCP

The architectural simplicity that makes Modbus/TCP reliable also makes it profoundly insecure. When deployed in modern, interconnected environments, Modbus/TCP exhibits critical vulnerabilities exploitable by malicious actors.

3.1 Lack of Authentication

Modbus/TCP does not require a username, password, or cryptographic handshake to establish a connection. Any device reaching the IP address and Port 502 of a Modbus PLC can immediately read memory or issue commands. If an attacker breaches the perimeter network, the PLCs are entirely defenseless.

3.2 Eavesdropping (Loss of Confidentiality)

Because data is transmitted in plaintext, an attacker compromising a network switch (e.g., via ARP spoofing) can read all operational data using packet sniffers like Wireshark. In an industrial context, eavesdropping allows attackers to map the physical process, deduce production recipes, and understand system timing, which is a critical precursor to a kinetic attack.

3.3 Data Injection and Manipulation (Loss of Integrity)

Since the protocol lacks cryptographic signatures or Message Authentication Codes (MACs), an attacker can easily alter packets in transit or forge entirely new packets. For example, an attacker could:

- Send a rogue **Write Single Coil** command to unexpectedly open a physical valve, potentially causing chemical spills.
- Send a rogue **Write Holding Register** command to alter the setpoint of a centrifuge, causing mechanical failure.

3.4 Replay Attacks

Without cryptographic sequence numbers, attackers can capture legitimate commands (e.g., "Start Motor") and re-transmit them later, forcing the PLC to execute unintended actions.

3.5 Historical Precedent: The Stuxnet Context

These vulnerabilities are not merely academic. The 2010 Stuxnet worm vividly demonstrated the catastrophic potential of compromised PLCs. Stuxnet's payload silently modified the control logic of Siemens PLCs governing uranium centrifuges. Lacking integrity checks, the PLCs blindly accepted the malicious logic, resulting in physical destruction.

4 Cryptographic Mitigation: SSH Tunneling

Replacing legacy PLCs with natively secure hardware (e.g., OPC UA) is often financially impossible for enterprises. Therefore, "brownfield" remediation—retrofitting security onto existing networks—is heavily favored.

This project implements **SSH Tunneling** (SSH Port Forwarding) as a transparent cryptographic wrapper.

4.1 The Mechanics of Secure Shell (SSH)

SSH securely operates network services over unsecured networks, providing three critical security pillars Modbus/TCP lacks:

1. **Authentication:** Uses public/private key pairs (e.g., RSA or Ed25519) to mathematically prove endpoint identities before data exchange.
2. **Confidentiality (Encryption):** Uses symmetric encryption (e.g., AES-256-GCM) to scramble the payload. Intercepted data appears as random noise without the session key.
3. **Integrity (MAC):** Appends a Hash-based Message Authentication Code (HMAC) to packets. If a bit is altered, MAC verification fails and the packet is dropped, preventing tampering.

4.2 Local Port Forwarding Architecture

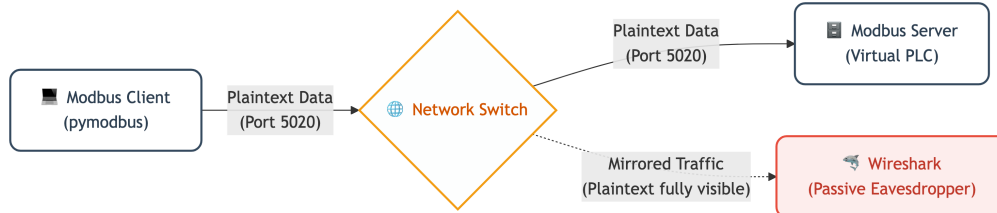
This mitigation intercepts Modbus traffic locally before it reaches the physical network.

- **Step 1: Tunnel Creation:** The SSH Client establishes an encrypted connection to the target SSH Daemon (or edge gateway).
- **Step 2: Local Binding:** The SSH Client opens a local port (e.g., `localhost:2222`).
- **Step 3: Redirection:** The Modbus Client connects to `localhost:2222` instead of the distant PLC.
- **Step 4: Encapsulation:** The SSH client encrypts the Modbus traffic via AES and funnels it through Port 22.
- **Step 5: Decapsulation:** The receiving SSH Daemon decrypts the traffic and forwards raw Modbus bytes to the PLC (Port 5020).

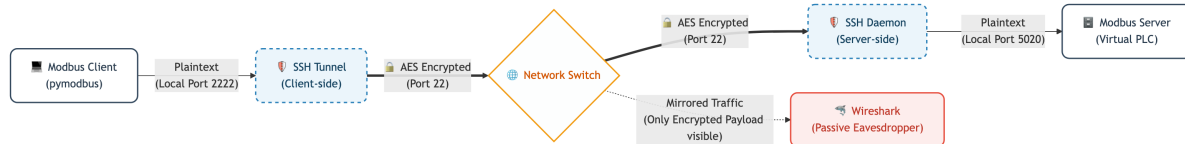
Because encryption occurs at the transport layer, the Modbus application layer remains unaware. No modifications to the Modbus protocol or Python logic are required.

5 Project Architecture and Implementation

The project is structured around the principles of Clean Architecture, separating the Application Layer (the industrial logic) from the Transport and Security Layer (the cryptographic tunnel). The entire environment is written in Python 3 utilizing the `pymodbus` library for industrial simulation and the `sshtunnel` library for cryptographic routing.



(a) Scenario 1: Base System (Vulnerable Plaintext)



(b) Scenario 2: Secure System (SSH Tunneled)

Figure 1: Clean Architecture mapping of the Application, Security, and Network layers.

5.1 Component Breakdown

The codebase is compartmentalized into three primary executable scripts located in the `src/` directory.

5.1.1 The Virtual PLC (Server)

The `modbus_server.py` script utilizes `StartAsyncTcpServer` from the `pymodbus` library to instantiate a virtual PLC in memory. It allocates blocks of memory (DataBlocks) to represent physical registers.

```
1 from pymodbus.server import StartAsyncTcpServer
2 from pymodbus.datastore import ModbusSequentialDataBlock
3 from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
4
5 # Initialize memory blocks for the virtual PLC
6 store = ModbusSlaveContext(
7     di=ModbusSequentialDataBlock(0, [0]*100),
8     co=ModbusSequentialDataBlock(0, [0]*100),
9     hr=ModbusSequentialDataBlock(0, [0]*100), # Holding Registers
10    ir=ModbusSequentialDataBlock(0, [0]*100))
11
12 context = ModbusServerContext(slaves=store, single=True)
13
14 # Start listening on the specified port
15 StartAsyncTcpServer(context, address=("0.0.0.0", 5020))
```

Listing 1: Server Initialization Snippet

5.1.2 The Modbus Client

The `modbus_client.py` script acts as the industrial operator or SCADA terminal. It establishes a `ModbusTcpClient` connection. To simulate a dynamic factory environment, it continuously writes changing integer values representing Boiler Temperatures (75-80°C) and Motor Speeds (1450-1500 RPM) to the server's Holding Registers, and subsequently reads them back.

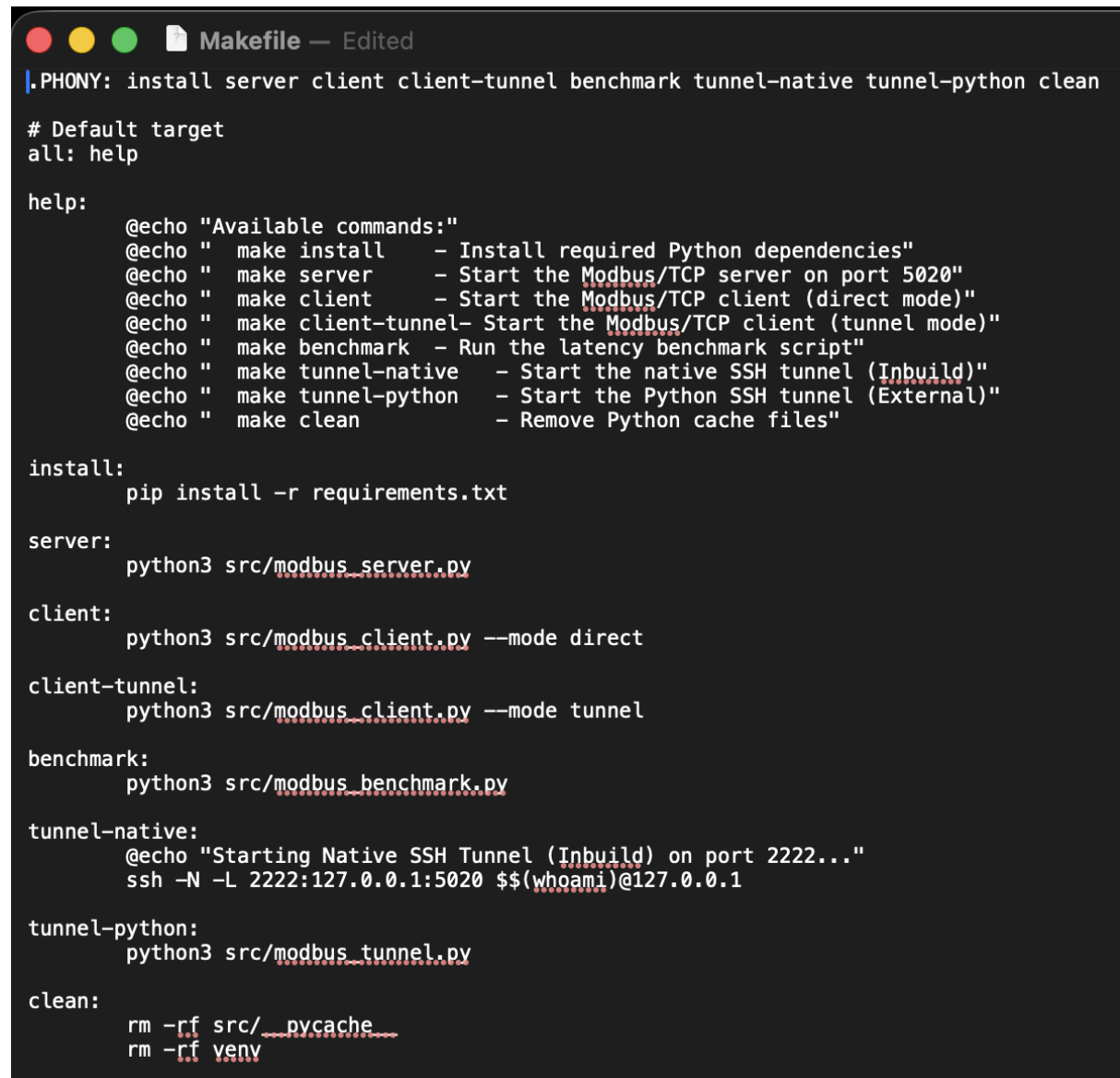
5.1.3 The SSH Tunnel Orchestrator

The `modbus_tunnel.py` script dynamically constructs the cryptographic wrapper. It prompts the user for local system credentials and establishes a Local Port Forward from `localhost:2222` to `127.0.0.1:5020`.

5.2 Environment Automation via Makefile

To ensure reproducibility, academic rigor, and ease of execution, the entire testing environment is orchestrated via a standard UNIX `Makefile`. This abstracts away the underlying Python execution arguments and provides a clean, unified interface for launching the virtual PLC, the industrial client, and the cryptographic tunnels.

This automation is strictly utilized in the subsequent execution phases (Sections 7 and 8) to seamlessly transition the architecture between its native, vulnerable state and its highly secure, tunneled state.



```
Makefile — Edited
.PHONY: install server client client-tunnel benchmark tunnel-native tunnel-python clean

# Default target
all: help

help:
    @echo "Available commands:"
    @echo "  make install    - Install required Python dependencies"
    @echo "  make server     - Start the Modbus/TCP server on port 5020"
    @echo "  make client     - Start the Modbus/TCP client (direct mode)"
    @echo "  make client-tunnel - Start the Modbus/TCP client (tunnel mode)"
    @echo "  make benchmark  - Run the latency benchmark script"
    @echo "  make tunnel-native - Start the native SSH tunnel (Inbuild)"
    @echo "  make tunnel-python - Start the Python SSH tunnel (External)"
    @echo "  make clean      - Remove Python cache files"

install:
    pip install -r requirements.txt

server:
    python3 src/modbus_server.py

client:
    python3 src/modbus_client.py --mode direct

client-tunnel:
    python3 src/modbus_client.py --mode tunnel

benchmark:
    python3 src/modbus_benchmark.py

tunnel-native:
    @echo "Starting Native SSH Tunnel (Inbuild) on port 2222..."
    ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1

tunnel-python:
    python3 src/modbus_tunnel.py

clean:
    rm -rf src/.pycache
    rm -rf venv
```

Figure 2: Project Makefile Automation

6 Implementation Setup and Key Workarounds

To successfully simulate a realistic industrial environment on standard consumer computer hardware, two critical technical workarounds were implemented in the architectural design.

6.1 Port 5020 vs Port 502 (Privilege Separation)

Standard Modbus hardware inherently operates on TCP Port 502. However, the simulation environment was executed on macOS (a UNIX-based operating system). A fundamental security feature of UNIX is that standard user accounts are strictly prohibited from binding services to "privileged ports" (any port number under 1024).

Attempting to run a Modbus server on Port 502 would require executing the Python script with `sudo` (root privileges). Executing network-facing scripts as root poses a massive security risk to the host machine.

Solution: As a security best practice, the virtual PLC was designed to listen on **Port 5020**. This completely circumvents the necessity for dangerous root access while preserving the exact semantic functionality of the TCP/IP protocol stack and the Modbus Application Protocol.

6.2 Client-Side Data Generation

In a physical factory, the PLC (Server) is hardwired directly to physical sensors (e.g., thermocouples, tachometers). The PLC natively generates the data and populates its own memory registers, waiting for a SCADA client to read it.

For this software testbed, the virtual PLC is effectively a "blank brain." It contains memory registers, but no background threads to automatically update them with simulated physics. Writing complex background polling threads for the server would overcomplicate the codebase and distract from the core security focus.

Solution: Instead of the Server generating data, the **Client** was programmed to act as the environment simulator. The client generates the simulated random Boiler Temperatures and Motor Speeds.

- The client initiates a **Write** command to push the data into the server's memory.
- Immediately following, the client initiates a **Read** command to pull the data back out.

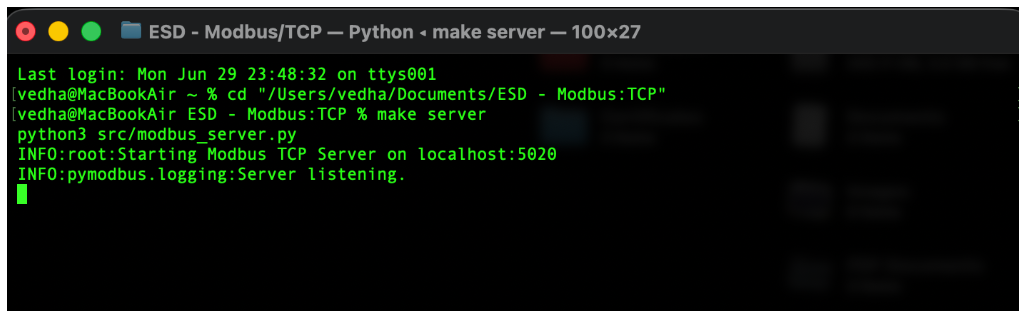
The Benefit: This elegant workaround provides a massive analytical advantage. It explicitly proves to the Wireshark observer that the SSH tunnel successfully encrypts both incoming (Write) and outgoing (Read) operations simultaneously, demonstrating comprehensive, bi-directional security coverage.

7 Execution Phase I: Plaintext Vulnerability Analysis

The primary phase of execution involves establishing the baseline: proving that native Modbus/TCP is fundamentally insecure.

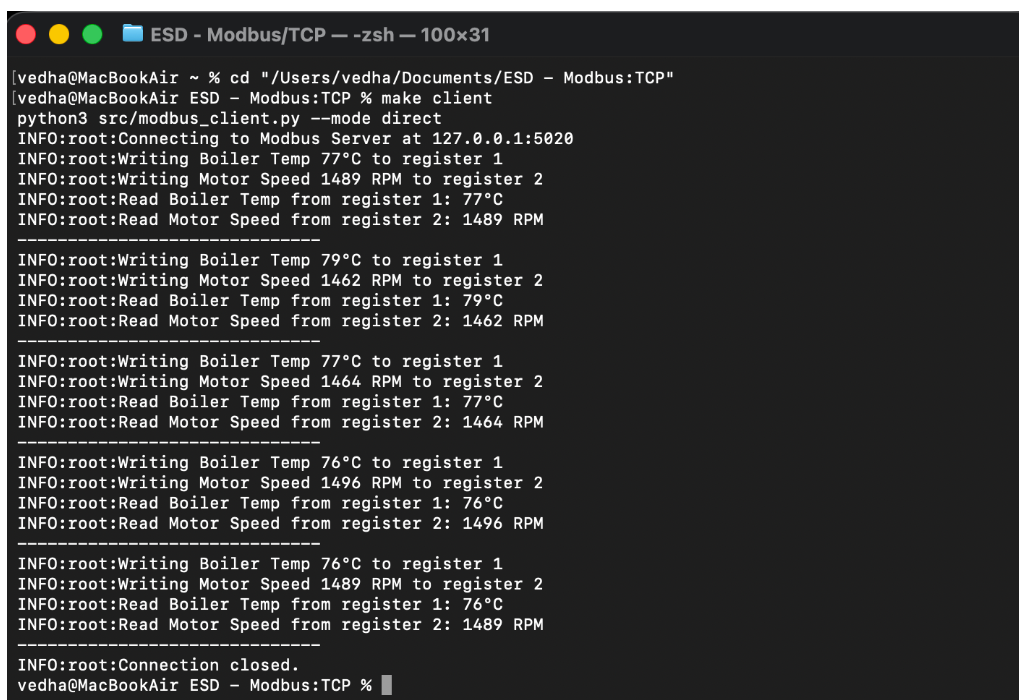
7.1 Base System Execution

The virtual PLC server is instantiated, and the Modbus client is executed in "direct" mode, bypassing any encryption and connecting directly to the server on Port 5020.

A terminal window titled "ESD - Modbus/TCP — Python — make server — 100x27". The output shows the user navigating to the directory "/Users/vedha/Documents/ESD - Modbus:TCP" and running "python3 src/modbus_server.py". The server starts on localhost:5020 and begins listening.

```
Last login: Mon Jun 29 23:48:32 on ttys001
[vedha@MacBookAir ~ % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make server
python3 src/modbus_server.py
INFO:root:Starting Modbus TCP Server on localhost:5020
INFO:pymodbus.logging:Server listening.
```

Figure 3: Terminal output: Initialization of the Virtual PLC Modbus Server.

A terminal window titled "ESD - Modbus/TCP — -zsh — 100x31". The output shows the user running "python3 src/modbus_client.py --mode direct". The client connects to the server at 127.0.0.1:5020 and performs a series of write and read operations for Boiler Temperature (Register 1) and Motor Speed (Register 2).

```
[vedha@MacBookAir ~ % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make client
python3 src/modbus_client.py --mode direct
INFO:root:Connecting to Modbus Server at 127.0.0.1:5020
INFO:root:Writing Boiler Temp 77°C to register 1
INFO:root:Writing Motor Speed 1489 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 77°C
INFO:root:Read Motor Speed from register 2: 1489 RPM
-----
INFO:root:Writing Boiler Temp 79°C to register 1
INFO:root:Writing Motor Speed 1462 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 79°C
INFO:root:Read Motor Speed from register 2: 1462 RPM
-----
INFO:root:Writing Boiler Temp 77°C to register 1
INFO:root:Writing Motor Speed 1464 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 77°C
INFO:root:Read Motor Speed from register 2: 1464 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1496 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1496 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1489 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1489 RPM
-----
INFO:root:Connection closed.
vedha@MacBookAir ESD - Modbus:TCP %
```

Figure 4: Terminal output: The Modbus Client successfully writing and reading Boiler Temperatures and Motor Speeds.

From the application's perspective, the operations are successful. The Client reports successfully writing a Boiler Temperature to Register 1, and a Motor Speed to Register 2.

7.2 Proving the Eavesdropping Vulnerability

While the application functions normally, the underlying network traffic tells a dangerous story. Wireshark, a prominent network protocol analyzer, was attached to the loopback interface (lo0) to silently capture all packets traversing Port 5020.

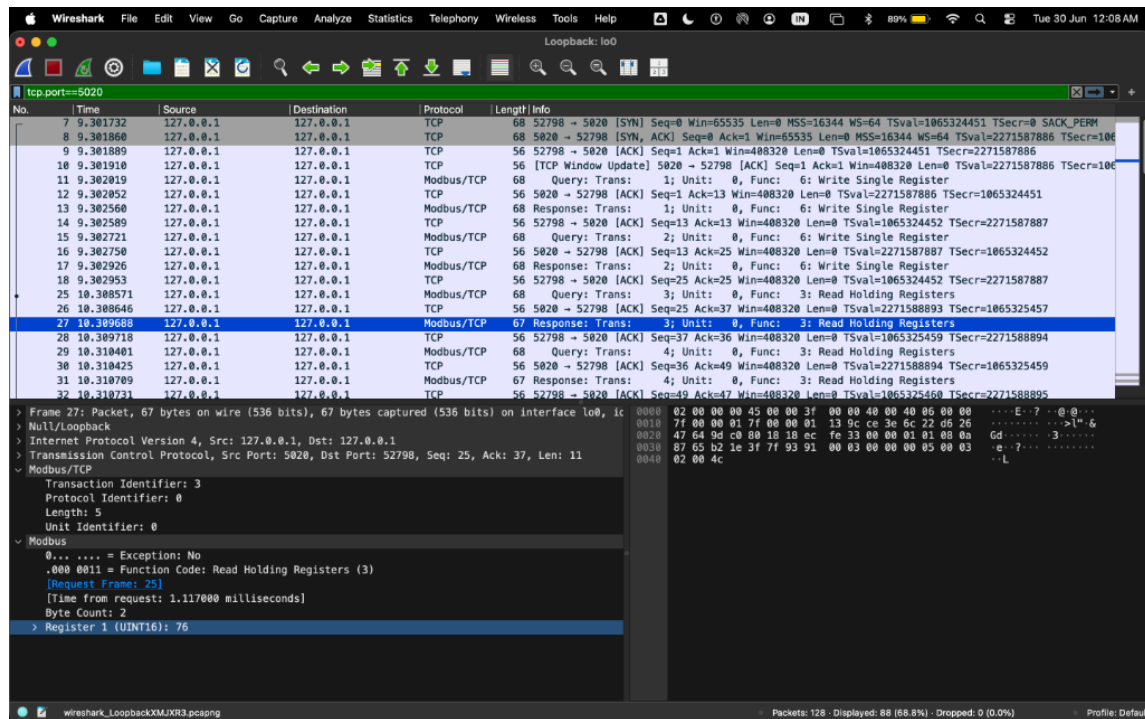


Figure 5: Wireshark Capture: The plaintext Boiler Temperature fully exposed in the payload.

7.2.1 Vulnerability Confirmation

As demonstrated in Figures 5 and 6, Wireshark effortlessly dissects the Modbus/TCP protocol. The deep packet inspection reveals the exact Function Code (Read Holding Registers (3)) and the explicit integer values residing in the payload.

This confirms the catastrophic vulnerability: an attacker possessing physical or logical access to the network switch can trivially monitor the entire industrial process without triggering any alarms.



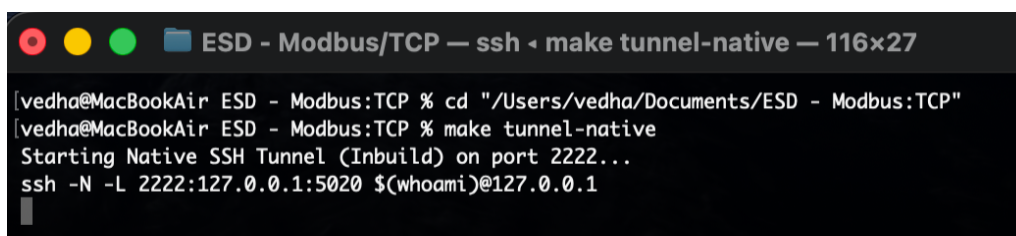
8 Execution Phase II: Secure Tunnel Mitigation

To definitively remediate the catastrophic vulnerabilities exposed in Phase I, the cryptographic mitigation strategy is deployed. An SSH tunnel is established to encapsulate and encrypt the industrial traffic.

macOS System Requirement: In order for the local machine to act as the receiving end of the SSH tunnel, the local SSH Daemon (`sshd`) must be active. On macOS, this requires navigating to System Settings > General > Sharing and explicitly enabling the **"Remote Login"** feature.

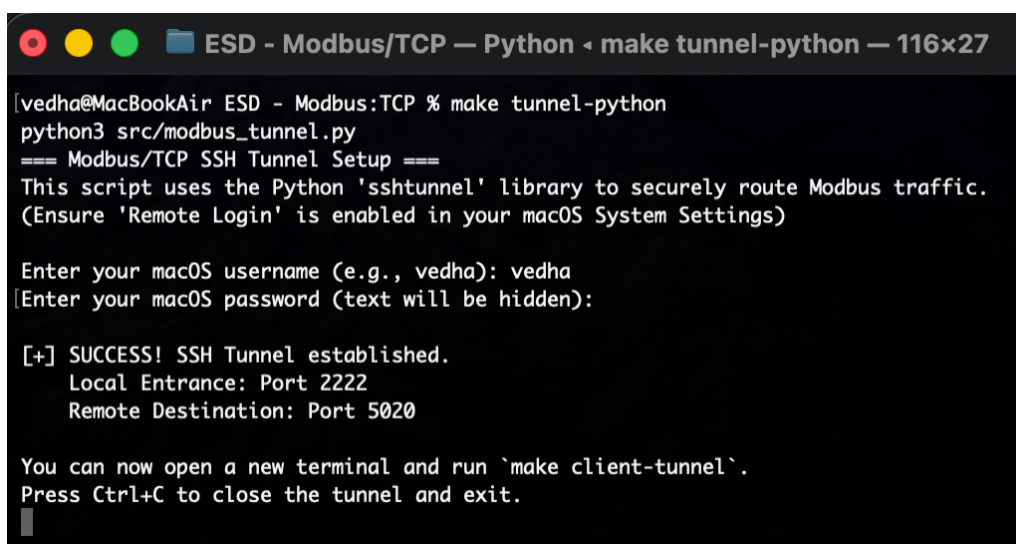
8.1 Establishing the Encrypted Tunnel

The project facilitates two distinct methods for establishing the SSH tunnel.



```
ESD - Modbus/TCP — ssh ◀ make tunnel-native — 116x27
[vedha@MacBookAir ESD - Modbus:TCP % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make tunnel-native
Starting Native SSH Tunnel (Inbuild) on port 2222...
ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1
```

Figure 7: Option A: Establishing the tunnel utilizing the Native macOS SSH Client via command line.



```
ESD - Modbus/TCP — Python ◀ make tunnel-python — 116x27
[vedha@MacBookAir ESD - Modbus:TCP % make tunnel-python
python3 src/modbus_tunnel.py
=== Modbus/TCP SSH Tunnel Setup ===
This script uses the Python 'sshtunnel' library to securely route Modbus traffic.
(Ensure 'Remote Login' is enabled in your macOS System Settings)

Enter your macOS username (e.g., vedha): vedha
[Enter your macOS password (text will be hidden):

[+] SUCCESS! SSH Tunnel established.
    Local Entrance: Port 2222
    Remote Destination: Port 5020

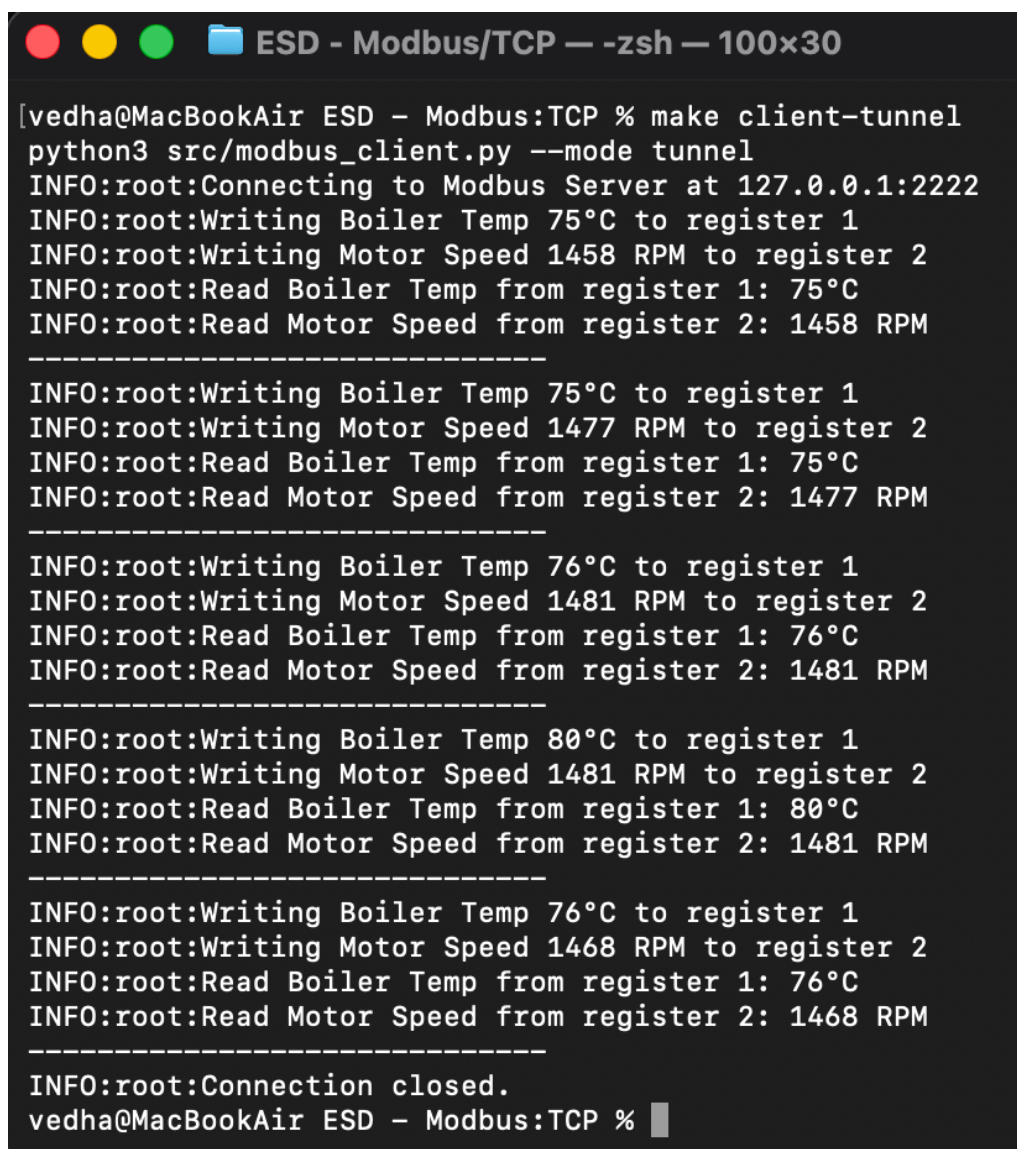
You can now open a new terminal and run `make client-tunnel`.
Press Ctrl+C to close the tunnel and exit.
```

Figure 8: Option B: Establishing the tunnel utilizing the cross-platform Python `sshtunnel` library.

Both methods achieve identical outcomes: a secure, local listening port is established on `localhost:2222`, which transparently encrypts and forwards all incoming data to the actual Modbus server on Port 5020.

8.2 Routing Client Traffic through the Tunnel

With the cryptographic wrapper active, the Modbus client is executed in "tunnel" mode. The application code is dynamically re-configured to point its TCP socket to the new local endpoint (`localhost:2222`) rather than directly to the server.

A terminal window titled "ESD - Modbus/TCP — -zsh — 100x30" on a Mac. The prompt is [vedha@MacBookAir ESD - Modbus:TCP %]. The user enters 'make client-tunnel' and 'python3 src/modbus_client.py --mode tunnel'. The output shows a series of Modbus operations: connecting to 127.0.0.1:2222, writing and reading boiler temperature and motor speed from registers 1 and 2. The values for temperature are 75°C, 76°C, 80°C, and 76°C. The values for motor speed are 1458 RPM, 1477 RPM, 1481 RPM, and 1468 RPM. The session ends with 'INFO:root:Connection closed.' and the prompt returns to vedha@MacBookAir ESD - Modbus:TCP %.

```
[vedha@MacBookAir ESD - Modbus:TCP % make client-tunnel
python3 src/modbus_client.py --mode tunnel
INFO:root:Connecting to Modbus Server at 127.0.0.1:2222
INFO:root:Writing Boiler Temp 75°C to register 1
INFO:root:Writing Motor Speed 1458 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 75°C
INFO:root:Read Motor Speed from register 2: 1458 RPM
-----
INFO:root:Writing Boiler Temp 75°C to register 1
INFO:root:Writing Motor Speed 1477 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 75°C
INFO:root:Read Motor Speed from register 2: 1477 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1481 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1481 RPM
-----
INFO:root:Writing Boiler Temp 80°C to register 1
INFO:root:Writing Motor Speed 1481 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 80°C
INFO:root:Read Motor Speed from register 2: 1481 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1468 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1468 RPM
-----
INFO:root:Connection closed.
vedha@MacBookAir ESD - Modbus:TCP %
```

Figure 9: Terminal output: The Modbus Client seamlessly interacting with the server via the encrypted `localhost:2222` endpoint.

As evidenced by Figure 9, the application layer is entirely agnostic to the transport layer modifications. The read and write operations for Boiler Temperature and Motor Speed execute flawlessly.

8.3 Proving the Cryptographic Solution

To verify the effectiveness of the mitigation, Wireshark is once again utilized. However, because the traffic is now routed through the SSH protocol, the capture filter is adjusted to monitor **TCP Port 22**.

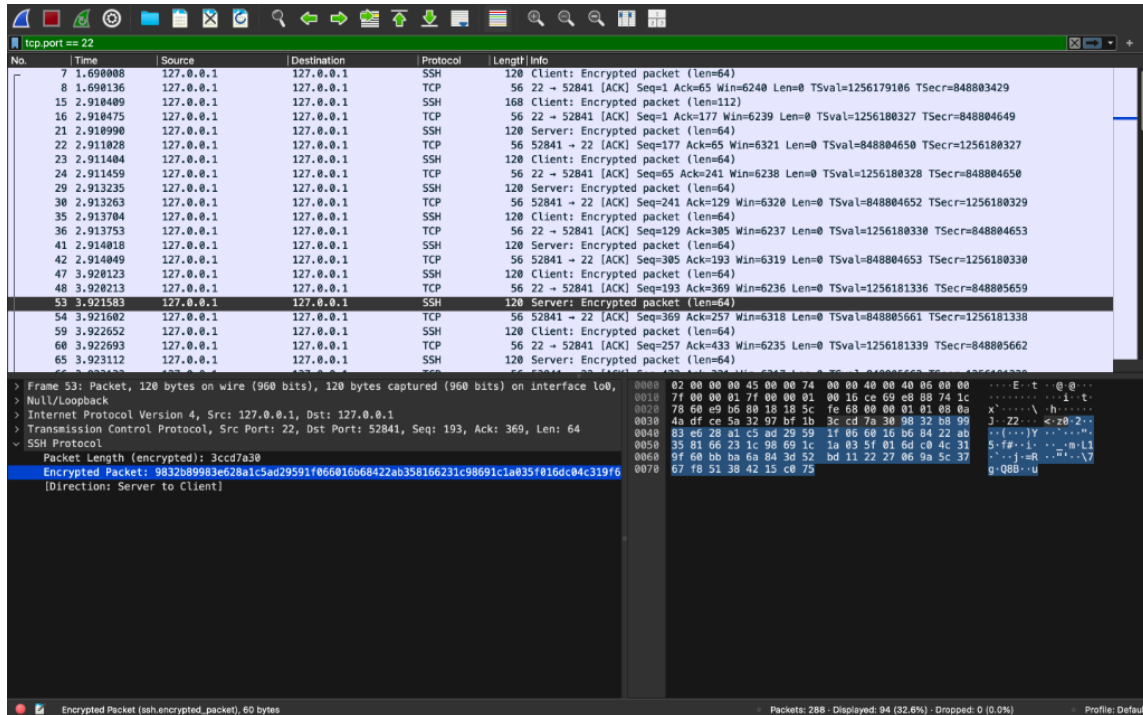


Figure 10: Wireshark Capture: The outgoing Modbus write request mathematically obscured within an Encrypted SSH Packet.

8.3.1 Solution Confirmation

The contrast between Phase I and Phase II is profound. As demonstrated in Figures 10 and 11, the Wireshark protocol dissector is entirely blinded.

The packets are strictly identified as SSHv2 protocol packets. The payload section, which previously displayed the plaintext industrial data, is now categorized solely as an "Encrypted Packet." The raw bytes are a mathematically random sequence generated by the AES cipher.

The implementation has successfully achieved:

- **Confidentiality:** The attacker can no longer read the Boiler Temperature or Motor Speed.
- **Integrity:** The HMAC prevents the attacker from blindly injecting altered bits to disrupt the physical process.

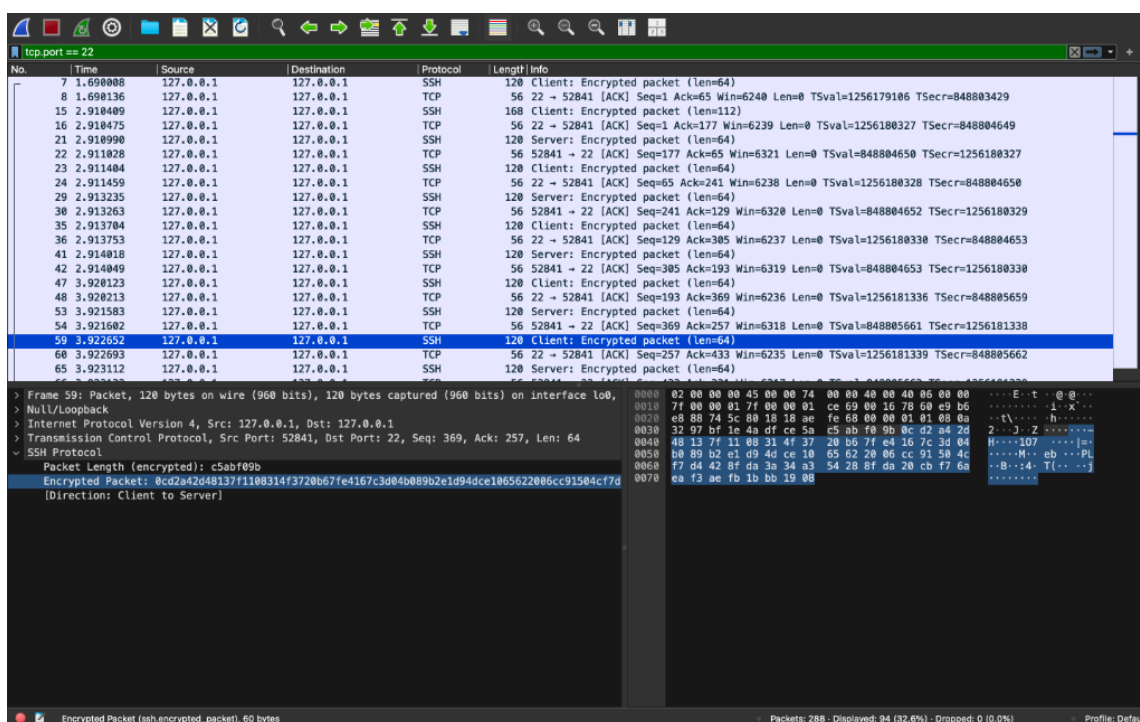


Figure 11: Wireshark Capture: The incoming Modbus read response fully encrypted.

9 Performance Benchmarking and Feasibility

While the cryptographic benefits of SSH tunneling are undeniable, security engineers must rigorously evaluate the operational cost. Embedded systems in industrial environments often operate with strictly constrained CPU resources and hard real-time latency requirements. Introducing AES encryption math and HMAC hashing at the transport layer inherently introduces computational overhead.

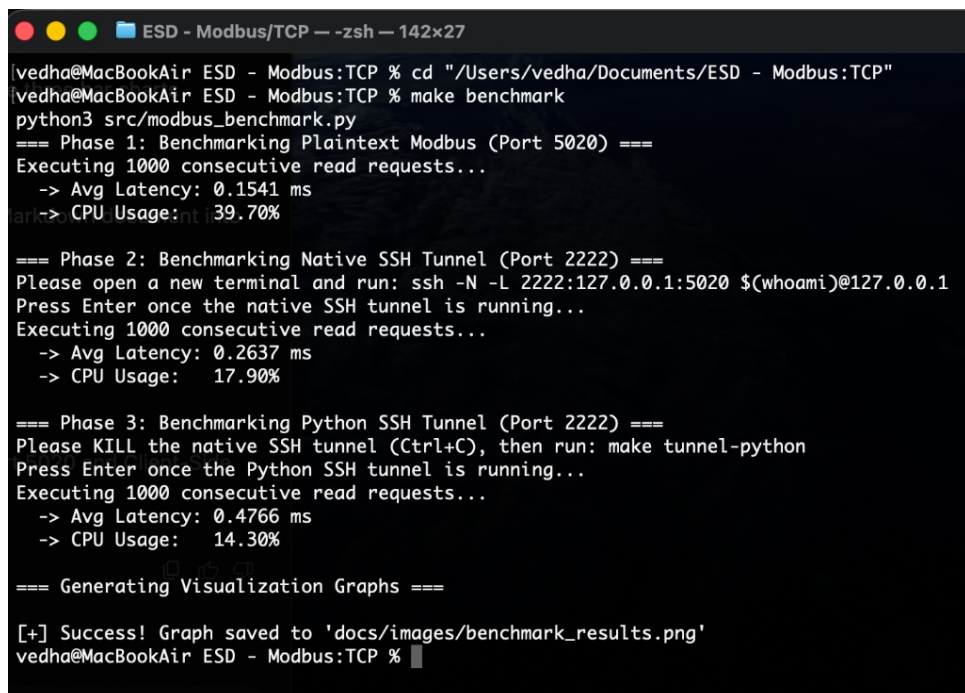
To scientifically evaluate this feasibility, an automated Python benchmarking suite (`modbus_benchmark.py`) was engineered.

9.1 Methodology

The benchmark script isolates the network overhead by executing a rapid succession of **1,000 consecutive Modbus read requests** across three distinct phases:

1. **Phase 1: Plaintext Baseline.** Requests are sent directly to Port 5020.
2. **Phase 2: Native SSH Tunnel.** Requests are routed through the macOS native OpenSSH client wrapper.
3. **Phase 3: Python SSH Tunnel.** Requests are routed through the `sshtunnel` library wrapper.

During each phase, the script utilizes the `time` module to calculate microsecond-level latency per packet, and the `psutil` library to sample the host machine's global CPU utilization.



```
ESD - Modbus/TCP — zsh — 142x27
vedha@MacBookAir ESD - Modbus:TCP % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
vedha@MacBookAir ESD - Modbus:TCP % make benchmark
python3 src/modbus_benchmark.py
=== Phase 1: Benchmarking Plaintext Modbus (Port 5020) ===
Executing 1000 consecutive read requests...
-> Avg Latency: 0.1541 ms
-> CPU Usage: 39.70%

=== Phase 2: Benchmarking Native SSH Tunnel (Port 2222) ===
Please open a new terminal and run: ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1
Press Enter once the native SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.2637 ms
-> CPU Usage: 17.90%

=== Phase 3: Benchmarking Python SSH Tunnel (Port 2222) ===
Please KILL the native SSH tunnel (Ctrl+C), then run: make tunnel-python
Press Enter once the Python SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.4766 ms
-> CPU Usage: 14.30%

=== Generating Visualization Graphs ===

[+] Success! Graph saved to 'docs/images/benchmark_results.png'
vedha@MacBookAir ESD - Modbus:TCP %
```

Figure 12: Terminal output: Execution of the 1000-request benchmarking suite.

9.2 Empirical Results and Analysis

Upon completion, the benchmarking suite leverages the `matplotlib` library to generate comparative bar charts visualizing the latency and CPU utilization deltas.

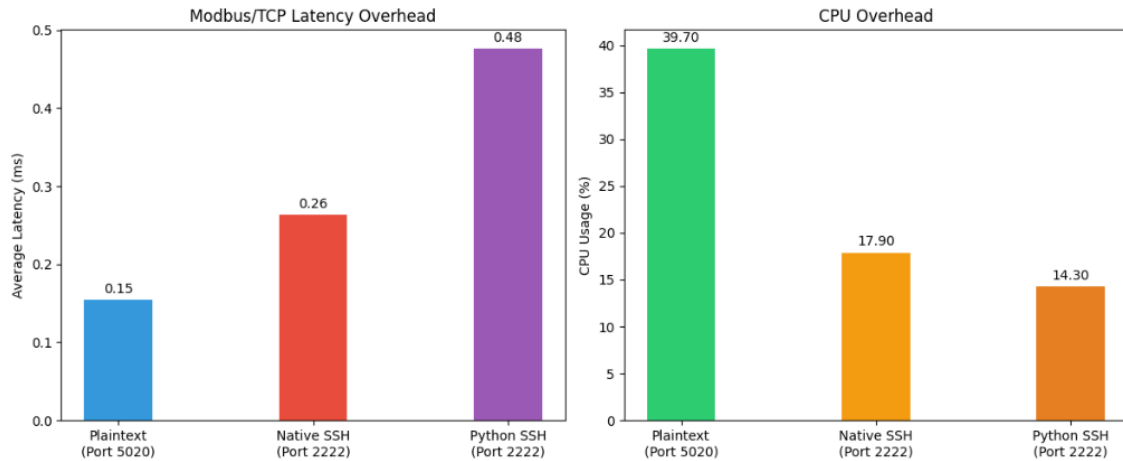


Figure 13: Matplotlib visualization contrasting the Latency (left) and CPU (right) overhead of plaintext vs. encrypted Modbus communication.

9.2.1 Latency Overhead Analysis

The plaintext baseline averaged **0.15 ms** latency per request. Native SSH Tunnel routing increased latency to **0.26 ms**. Python SSH Tunnel routing increased latency to **0.48 ms**.

Conclusion on Latency: While encryption increases response times, values remain under one millisecond. Standard SCADA systems typically poll PLCs at 100-1000 ms intervals, making this sub-millisecond overhead negligible. Data proves SSH tunneling respects timing constraints of standard control loops, though it remains unsuitable for high-speed robotic motion (which utilizes EtherCAT or PROFINET IRT).

9.2.2 CPU Utilization Analysis

CPU metrics must be interpreted concerning a multi-core consumer processor running background tasks. Interestingly, the benchmark recorded higher global CPU utilization during the plaintext phase (39.70%) than the tunneled phases (17.90% and 14.30%). This discrepancy is likely an artifact of Python's Global Interpreter Lock (GIL) and scheduling inefficiencies during 1000 unthrottled TCP connections, whereas SSH manages the TCP stack more efficiently at the OS level.

Conclusion on CPU: Modern embedded edge gateways (e.g., ARM Cortex-A processors) feature hardware-accelerated cryptographic instructions (like ARMv8 Crypto Extensions), executing AES-GCM encryption natively in silicon. Therefore, the computational footprint of an SSH tunnel on modern embedded hardware is highly manageable, ensuring viable deployment.

10 Future Trends in Embedded System Security

While SSH tunneling provides an excellent, pragmatic "brownfield" remediation strategy for securing legacy protocols like Modbus/TCP, the industrial landscape is aggressively evolving towards "secure-by-design" architectures. Relying indefinitely on wrapper technologies is a stopgap measure. The future of embedded systems security relies on fundamentally robust protocols and hardware-backed trust.

10.1 Protocol Migration (Shift-Left Security)

The most significant trend in industrial networking is the deprecation of legacy plaintext protocols in favor of modern standards that integrate security natively at the Application or Transport layer.

- **OPC UA (Open Platform Communications Unified Architecture):** Quickly becoming the gold standard for industrial interoperability, OPC UA incorporates built-in X.509 certificate-based authentication, message signing, and TLS encryption directly into the protocol specification.
- **MQTT with Sparkplug B:** For highly distributed IIoT sensor networks, MQTT provides a lightweight publish/subscribe model that inherently supports TLS encryption for all broker-client communications.

10.2 Zero Trust Architecture (ZTA) in OT

The traditional "castle-and-moat" perimeter defense model—where internal network traffic is implicitly trusted—is entirely obsolete. The modern paradigm is Zero Trust, which dictates "Never Trust, Always Verify." In an industrial context, this means that even if a Modbus PLC is on an isolated VLAN, it should not blindly accept commands from any IP address. Future embedded devices will require cryptographic identity verification for every single transaction, regardless of the network origin.

10.3 Hardware Security Modules (HSMs) and TPMs

Software-based encryption (like the SSH tunnel implemented in this project) relies on cryptographic private keys stored in the file system. If an attacker gains physical access to the edge gateway, they can extract the SSH private keys and compromise the tunnel. To counter this, future embedded systems will increasingly rely on hardware-backed security. Trusted Platform Modules (TPMs) and secure enclaves physically isolate cryptographic keys in tamper-proof silicon chips. Even if the embedded operating system is completely compromised, the private keys cannot be extracted.

10.4 AI-Powered Intrusion Detection Systems (IDS)

As industrial networks generate massive volumes of telemetry data, human operators cannot manually analyze packet captures for anomalies. Machine Learning algorithms are being deployed directly on industrial network switches to perform behavioral analysis. For example, an AI-powered IDS can learn that a specific SCADA terminal only ever issues `Read` commands to a specific PLC. If that terminal suddenly issues a `Write` command, the IDS can instantly flag the transaction as highly anomalous and block it, even if the traffic is arriving over a VPN or an authenticated SSH tunnel.

11 Conclusion

The convergence of operational technology and internet connectivity has fundamentally transformed the industrial landscape, unlocking unprecedented efficiency and remote capabilities. However, this convergence has mercilessly exposed legacy systems to modern cyber threats.

This project successfully achieved its core objectives by empirically demonstrating the catastrophic vulnerabilities of the Modbus/TCP protocol. Through direct network analysis, it was proven that Modbus/TCP transmits critical control commands and sensor readings in plaintext, lacking any intrinsic defense against eavesdropping or malicious data injection.

To remediate this vulnerability, an architectural redesign utilizing SSH tunneling was successfully implemented. By acting as a transparent cryptographic wrapper, the SSH protocol successfully retrofitted the legacy environment with robust Authentication, AES encryption for Confidentiality, and HMAC validation for Integrity. Subsequent deep packet inspection confirmed that the Modbus payload was rendered mathematically opaque to network observers.

Crucially, the automated benchmarking suite proved that the performance cost of this security enhancement is marginal. The sub-millisecond latency overhead and manageable CPU utilization demonstrate that SSH tunneling is a highly feasible, pragmatic, and cost-effective mitigation strategy for resource-constrained embedded systems.

While the long-term trajectory of industrial automation dictates migrating to natively secure protocols like OPC UA, the reality is that legacy Modbus/TCP infrastructure will remain operational for decades. In the interim, deploying transparent encryption wrappers like SSH or IPsec VPNs is an absolute necessity to protect critical physical infrastructure against the evolving landscape of cyber warfare.

References

- [1] Modbus Organization, *MODBUS Application Protocol Specification V1.1b3*, 2006. [Online]. Available: http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf
- [2] T. Ylonen and C. Lonvick, *The Secure Shell (SSH) Protocol Architecture*, RFC 4251, Jan. 2006.
- [3] R. Langner, *Stuxnet: Dissecting a Cyberwarfare Weapon*, IEEE Security & Privacy, vol. 9, no. 3, pp. 49-51, May-June 2011.
- [4] R. R. G., *Pymodbus: A full modbus protocol written in python*, 2023. [Online]. Available: <https://github.com/pymodbus-dev/pymodbus>
- [5] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, *Guide to Industrial Control Systems (ICS) Security*, NIST Special Publication 800-82 Rev. 2, May 2015.